

Governed Human-AI Collaboration Across the Software Development Lifecycle: A Process Framework and Its Reference Implementation

Tomás dos Santos Taborda
Instituto Superior Técnico, Universidade de Lisboa
Lisbon, Portugal
tomassantostaborda@tecnico.ulisboa.pt

ABSTRACT

Generative Artificial Intelligence and Large Language Models have moved from autocomplete assistants to active contributors across the Software Development Life Cycle, yet adoption has outpaced method: a majority of functionally correct AI-generated code has been reported to carry security defects, and measured delivery bottlenecks shift toward review rather than disappearing. A Systematic Literature Review conducted for this work, following Kitchenham’s guidelines over 646 retrieved articles, confirms this pattern and identifies the absence of structured, repeatable guidance for integrating AI tools into professional software workflows as the field’s central unresolved problem. This dissertation addresses that gap through the Design Science Research Methodology. It proposes a process-oriented framework for governed Human-AI collaboration that treats AI systems as first-class collaborators whose contributions must be explicitly bounded, validated and traced: six lifecycle phases with phase-specific quality gates, responsibilities expressed as functions rather than job titles, three governance metrics, and a persistent, versioned specification that anchors every generated artefact to its originating requirement. The framework is instantiated as Apex, a working web application that operationalises each phase, gate and artefact against a real project management backend. Evaluation combines application of the artefact in a real-world project, standardised instruments assessing perceived usability and cognitive workload, semi-structured interviews with practitioners and Agile experts, and an analytical assessment against defined success criteria.

[PLACEHOLDER - WAITING ON DATA:] This abstract is a scaffold written before the usability study has been run. The evaluation and results sentences above describe the planned instruments, not obtained findings; a one-sentence headline result belongs here once the study is complete, per the note at §6-§8 below.

KEYWORDS

Artificial Intelligence, Large Language Models, Software Development Life Cycle, Design Science Research, Human-AI Collaboration, Software Process, Governance

1 MOTIVATION AND PROBLEM STATEMENT

Adoption of AI coding assistance is close to universal while trust in it is not, and the two are moving in opposite directions. The 2025 Stack Overflow Developer Survey reports 84% of developers using or planning to use AI tools, up from 76% the previous year, while the share trusting the accuracy of that output fell from 40% to 29%,

with more respondents actively distrusting AI output (46%) than trusting it (33%) [26]; industry telemetry places daily use higher still, at around 90% of developers [8]. That distrust is empirically warranted: benchmarking a leading coding agent on real-world tasks, Zhao et al. found 61% of solutions functionally correct but only 10.5% secure [30], and an assessment across more than one hundred models found 45% of generated samples introduced an OWASP Top 10 vulnerability, rising to 72% for Java, with newer and larger models improving on functional correctness while security performance stayed flat [28]. Productivity gains are similarly conditional rather than automatic: an early controlled experiment reported a 55.8% speed-up, though it remains unpublished [22], while a randomised controlled trial of sixteen experienced developers on their own mature repositories found they took 19% *longer* with AI assistance while estimating afterwards that it had made them 20% faster [5]; at organisational scale, telemetry from 1,255 teams associates high AI adoption with 98% more pull requests merged but 91% longer review times and no significant correlation with delivery outcomes [10].

The value of AI assistance in software engineering is conditional on the process that surrounds it, rather than being a property of the model alone. Capability without governance redistributes effort rather than reducing it, and can degrade quality attributes that are expensive to recover later.

A Systematic Literature Review conducted for this work (Section 2) confirms this reading directly: while the corpus consistently reports meaningful benefits from AI and LLM use, it equally consistently reports that organisations lack established, clear methodologies to shape the responsible use of these tools within professional software development [4, 9, 13, 16]. Existing methodologies do not close this gap: Waterfall, Scrum, Kanban and the Agile Manifesto were designed for development environments in which every contributor is human, and lack explicit structures for governing AI participation, bounding the context a model may act upon, or allocating responsibility over generated artefacts [29].

The research problem addressed in this work is the absence of structured, repeatable guidance and standardised evaluation practices for integrating AI and LLM-based tools across the SDLC while preserving human oversight, software quality and security, and organisational accountability.

This dissertation decomposes the objective of designing, instantiating, demonstrating and evaluating a framework that addresses this problem into four Dissertation Research Questions (DRQ1-4): how AI-supported activity can be explicitly mapped to SDLC

phases (DRQ1); which governance mechanisms keep contributions traceable, validated and accountable (DRQ2); to what extent the resulting constructs can be operationalised in a working system (DRQ3); and whether applying the framework provides practitioners with usable, cognitively sustainable support in a real setting (DRQ4).

2 RESEARCH METHODOLOGY

Two complementary methodologies structure this work. A Systematic Literature Review, following Kitchenham’s three-phase protocol (planning, conducting, reporting), retrieved 646 articles and synthesised the corpus around five research questions covering benefits, consequences, integration methodologies, stakeholder perspectives, and adopting industry sectors (RQ1-RQ5, reported in full in the dissertation’s Systematic Literature Review chapter). The Design Science Research Methodology proposed by Peffers et al. structures the design, construction, demonstration and evaluation of the artefact across its six activities: problem identification, definition of objectives, design and development, demonstration, evaluation, and communication. The two methodologies are sequential rather than parallel: the review’s findings are the evidence base the problem statement and the framework’s design principles are grounded against, and DSRM’s iteration loop is not strictly linear, since feedback gathered while constructing the reference implementation fed back into the design of the framework itself.

3 RELATED WORK: FINDINGS OF THE LITERATURE REVIEW

The review’s findings on methodology (RQ3) are the most direct grounding for this dissertation’s proposal. Current industry practice is not built around full automation but around control: the most prevalent documented methods, In-Context Learning, Prompt Engineering and Chain-of-Thought Prompting, all aim to shape and constrain AI behaviour before code is generated, and the Human-In-The-Loop paradigm keeps developers directly responsible for reviewing, validating and approving all outputs. Multiple emerging frameworks, including Agile-based human-AI collaboration models, indicate the field is actively searching for structured and repeatable integration patterns, but no universally accepted approach has yet emerged [1]. Findings on benefits (RQ1) and consequences (RQ2) sharpen the tension the framework must resolve: reported benefits extend across the entire SDLC rather than code generation alone, but they are inseparable from risks that directly undermine the same benefit, chiefly reduced code quality, security exposure, and eroded traceability. Findings on stakeholder opinion (RQ4) converge across developers and organisations on the same trade-off: enthusiasm for productivity gains alongside concern about reliability and hallucination, and a shared insistence that AI augment expertise under a controlled level of trust rather than substitute for it.

The closest published methodological precedent is Amazon Web Services’ AI-Driven Development Lifecycle, which organises work into Inception, Construction and Operations, replaces sprints with short cycles called Bolts, and replaces epics with Units of Work [2]; its vendor-reported productivity outcomes are unverified and carry no evidential weight here, but its process vocabulary is adopted

directly. The closest engineering practice is spec-driven development, in which a persistent, versioned specification rather than the source code itself is the artefact both developers and AI agents work against, tracked by Thoughtworks’ Technology Radar as an emerging technique [27] and recently framed in a not-yet-peer-reviewed preprint as a response to a productivity-reliability paradox in AI-augmented development [11].

4 PROPOSED FRAMEWORK

The framework is a method in the Design Science Research sense: a set of phases, hats, artefacts and decision rules prescribing how a team organises AI participation across the SDLC, stated at the level of process rather than tooling so it can be assessed and adopted independently of any single tool stack. It deliberately excludes model training or fine-tuning, model architecture, and infrastructure-level MLOps, since these govern the lifecycle of a model as a deployed asset, whereas this framework governs the lifecycle of software a model helped write.

4.1 Design Principles

Eight principles, each grounded in the evidence of Sections 1 and 3 rather than asserted as preference, govern every design decision: Human-in-the-Loop by Design; AI as Augmentation, Not Substitution; Process-Level Governance of AI Usage; SDLC-Wide and Phase-Aware Integration; Hats, Not People; Explicit Responsibility and Accountability; Risk-Proportional Validation and Control; and Spec-Anchored Continuity. The last of these instantiates spec-driven development (Section 3) rather than inventing it, and is the mechanism argued in Section 4.4.

4.2 Lifecycle Phases

The framework organises work into six phases, preceded by a Strategic Alignment stage that decides whether a Unit of Work should exist at all. Table 1 summarises, for each, the permitted role of AI, the accountable hat, and the control that must pass before exit; each phase consumes the output of the one before it. The model departs from AI-DLC’s three-phase Inception/Construction/Operations split at two points, both argued from the evidence in Section 1 rather than announced: a decoupled Testing phase, because tests derived by the same model from the same specification in the same working context confirm internal consistency rather than external correctness, and a distinct Maintenance phase with a diagnostic firewall, because a post-deployment signal may be a genuine defect, an unspecified request, or evidence the specification itself was wrong, and treating all three as defects is what erodes a specification over time. Maintenance’s exit is always a governed loop back into a named earlier phase rather than a patch applied in place, which is the structural difference from a Waterfall lifecycle with a maintenance tail appended to it.

Table 1: The six lifecycle phases: condensed from the full phase-mapping table of the framework chapter.

Phase	Role of AI	Exit control
Discovery & Requirements	Drafts stories, compiles approved scenarios into Gherkin	Discovery Gate (the Trio)
Design, Prototyping & Architecture	Proposes UI structure and a technical specification	Design Gate (Design Lead, Tech Lead)
Implementation	Generates and refactors code inside a Bolt	Implementation Gate (Developer)
Testing	Drafts end-to-end BDD scripts from locked Gherkin	Testing Gate (QA)
Deployment	Generates infrastructure delta only when required	Deployment Gate (deploying hat)
Maintenance	Reconstructs a narrow context to diagnose one signal	Diagnostic firewall, governed loop-back

4.3 Roles as Hats

Every role named is a hat, a function the framework requires to be performed rather than a person who must be hired; one individual may wear several, and on a small team an unfilled hat’s responsibility falls to whoever performs the adjacent work. The Trio (Product Owner, Tech Lead and Design Lead hats) is inspired by the Squad concept popularised in the Spotify Model [18], cited alongside its own documented limits: it was a description of one company at one point in time rather than a blueprint [19], and a first-hand account names matrix ambiguity and autonomy without accountability as failure modes it never fully resolved even internally [20]. The framework answers both directly, the first through the Tech Lead hat’s pre-Bolt architectural lock, the second by separating logistics authority (Bolt Master hat) from technical authority (Tech Lead hat) and attaching every gate to a named function; the treatment of team cognitive load as an explicit design constraint [25] is the more rigorous vocabulary for the same argument, and the Project Manager hat sits outside the Squad closer to the single-threaded-leader pattern Amazon adopted once its earlier structure did not survive scale [7].

4.4 Governance Mechanisms and Quality Gates

A Bolt is the smallest execution iteration the framework recognises, the implementation of one task inside one User Story, measured in hours; the term is adopted from AI-DLC [2], and what the framework adds is the obligation sequence around it, since a short cycle without a fixed reference is simply a fast way to accumulate divergence. Spec-Anchored Continuity requires every AI-generated artefact to carry forward the context of previously validated phases through a persistent, versioned specification split into a global layer (architecture, technology-stack decisions, security policy) and a local layer (a story’s locked Gherkin and technical specification); the distinction argued for this mechanism is that narrowing a model’s working context is a control rather than a convenience, since an unbounded context is the specific condition under which a model produces a confident and substantively wrong change [30]. Five

quality gates, each with a named owning hat, a specific body of required evidence, and a defined consequence on failure, govern Discovery, Design, Implementation, Testing and Deployment; failure never discards work, it returns it to a named earlier activity with the evidence attached. Three governance metrics, Bolt Cycle Time, Context Traceability Rate, and AI Defect Escape Rate, are expressed in vocabulary continuous with the NIST AI Risk Management Framework’s Govern-Map-Measure-Manage loop [21] and ISO/IEC 42001’s plan-do-check-act vocabulary [17], a claim about shared language only; no conformity assessment against either standard has been performed. Grouping acceptance criteria at the Unit of Work level rather than requirement by requirement is independently supported: across 107 requirements, an epic-organised generation pipeline outperformed a requirement-aligned baseline on expert-rated correctness (4.61 against 4.14 of 5) and completeness (4.31 against 3.50) [24].

4.5 Replacing Sprint Cadence, and Positioning Against Alternatives

The framework replaces epics, sprints and sprint backlogs with continuous flow, on the ground that a fixed-length iteration is a mechanism for a world in which implementation is the binding constraint, and the evidence in Section 1 shows that premise no longer holds where AI assistance is heavily used: the constraint moves to review rather than disappearing [5, 10]. Four alternatives are engaged directly rather than by omission: no framework at all, correct for a short-lived prototype but contradicted by the flat security-versus-capability trend already cited; AI-DLC adopted unmodified, whose two gaps in build/test coupling and post-deployment classification motivate the divergence rather than a rejection of it; unmodified Scrum or SAFe cadence, undermined specifically by developers misjudging their own speed under assistance [5]; and metrics-only governance, the strongest of the four, answered by the distinction that metrics measure whether governance is working while containing no mechanism themselves for what a model may act upon or where a post-deployment signal is routed [12].

5 REFERENCE IMPLEMENTATION: APEX

[PLACEHOLDER - WAITING ON DATA:] Waiting on Chapter 7 of the dissertation (currently a skeleton with structural placeholder markers, not on study results). Draft once written: (1) Apex’s positioning against a bare coding agent, namely that it governs whether generated code still answers its originating requirement rather than accelerating the writing of code; (2) a condensed version of the framework-construct-to-mechanism mapping table (phases, gates, Spec-Anchored Continuity as versioned specification files, Bolt tracking and its cycle-time metric) that answers DRQ3; (3) one or two sentences on the split-stack architecture (Next.js frontend, Python/FastAPI backend) and the single-writer concurrency model; (4) a one-paragraph summary of the practitioner-feedback-driven design history (Chapter 7’s Design History section), since it is evidence that evaluation fed back into design, not merely a changelog.

6 DEMONSTRATION, EVALUATION DESIGN, AND RESULTS

The evaluation strategy is fixed independently of its outcome: a demonstration in a real project together with the System Usability Scale [6] and the Raw NASA Task Load Index [14, 15] evaluate the instantiation (Apex, DRQ4), while semi-structured interviews with practitioners and with Agile and Scrum experts, together with an analytical assessment against stated success criteria, evaluate the framework itself (DRQ1, DRQ2). A favourable usability score for the tool is deliberately not treated as evidence that the framework is sound; the two artefacts are evaluated by different means for that reason.

[PLACEHOLDER - WAITING ON DATA:] This entire section is waiting on data that does not exist yet: the usability study (unmoderated SUS and per-task Raw NASA-TLX administration, per the dissertation's Chapter 9 instruments) has not been run, so there are no participant counts, no SUS or NASA-TLX scores, no interview themes, and no demonstration narrative to report. Do not draft prose here beyond the fixed design above until the study is complete. When it is, this section needs: participant counts and roles (the dissertation's Evaluation chapter already fixes the ethics/consent process, reproduce only the results-facing parts); SUS scores (distribution, not mean alone, given the expected small sample) with the Bangor et al. adjective scale [3] and Sauro and Lewis's curved grading [23]; NASA-TLX per-subscale profile, not the aggregate alone; interview themes with anonymised representative material and disconfirming evidence given equal prominence; and the analytical assessment table (criteria, three-level rating, justification).

7 DISCUSSION AND THREATS TO VALIDITY

[PLACEHOLDER - WAITING ON DATA:] Waiting on the results in §6. Once available: synthesise across instruments, treating disagreement (for instance a favourable usability score alongside interview scepticism about process overhead) as a finding rather than smoothing it over; answer DRQ1-DRQ4 explicitly, each grounded in the specific evidence that answers it (DRQ1/DRQ2 from interviews and the analytical assessment, DRQ3 from the Apex construct-mechanism mapping read against its implementation limitations, DRQ4 from SUS, NASA-TLX and the demonstration); and state the threats already identifiable in advance of any result, since they follow from the evaluation's design rather than its outcome: the author's dual role as designer of both artefacts and their evaluator; a small, non-random, single-organisation convenience sample; an unmoderated protocol that removes facilitator influence during sessions but not from the task script, questionnaire wording, or interview conduct, all authored by the same person; and instruments that measure perception rather than delivered software quality, with no completion-rate, time-on-task, or error-count data collected to partly answer that gap, a deliberate consequence of recording nothing during sessions rather than an oversight.

8 CONCLUSION

This dissertation contributes a systematic synthesis of 646 articles on AI and LLM use in software development; a process framework

for governed Human-AI collaboration that treats AI as a first-class, explicitly bounded collaborator across six lifecycle phases; Apex, a working reference implementation demonstrating the framework's governance constructs are implementable rather than merely describable; and an empirical evaluation combining real-world application, standardised usability and workload measurement, practitioner and expert interviews, and an analytical assessment.

[PLACEHOLDER - WAITING ON DATA:] Waiting on the results in §6-§7. Once available, replace this paragraph with a genuine synthesis: a one-paragraph headline answer to the overall research problem (grounded, not asserted); the honest limitations already known in advance (single organisational setting, short observation window precluding any claim about long-term maintainability, perception-based instruments) stated plainly rather than conceded under questioning; and, per the dissertation's Chapter 2 Communication activity, a note on dissemination (the SLR article and this extended abstract) alongside the defence itself.

REFERENCES

- [1] I. Ahmed, A. Aleti, H. Cai, A. Chatzigeorgiou, P. He, X. Hu, M. Pezzè, D. Poshyvanyk, and X. Xia. 2025. Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead. *ACM Transactions on Software Engineering & Methodology* 34, 5 (2025), 1–27. <https://doi.org/10.1145/3719006>
- [2] Amazon Web Services. 2025. AI-Driven Development Lifecycle (AI-DLC). Vendor methodology announced at AWS DevSphere 2025. Reported productivity figures are vendor-reported and not independently verified. <https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/>
- [3] A. Bangor, P. Kortum, and J. Miller. 2009. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies* 4, 3 (2009), 114–123.
- [4] L. Banh, F. Holldack, and G. Strobel. 2025. Copiloting the future: How generative AI transforms Software Engineering. *Information and Software Technology* 183 (2025). <https://doi.org/10.1016/j.infsof.2025.107751>
- [5] J. Becker, N. Rush, E. Barnes, and D. Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. arXiv:2507.09089, METR, preprint, not yet peer-reviewed. <https://arxiv.org/abs/2507.09089>
- [6] J. Brooke. 1996. SUS: A 'quick and dirty' usability scale. In *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland (Eds.). Taylor and Francis, 189–194.
- [7] C. Bryar and B. Carr. 2021. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press, New York, NY.
- [8] DORA and Google Cloud. 2025. State of AI-assisted Software Development 2025. Industry research report, not peer-reviewed. <https://dora.dev/dora-report-2025/>
- [9] Mar'i Farhanna, Nerisafitra Paramitha, Marianingsih Susi, Hanjaya Surya, Aisyah Yanti, Primatama Dea, and Sutaji Deni. 2025. Exploring the Use of Generative AI in Software Development: A Preliminary Study. In *E3S Web of Conferences*, Vol. 645. 04002. <https://doi.org/10.1051/e3sconf/202564504002>
- [10] Faros AI. 2025. The AI Productivity Paradox Report 2025. Industry telemetry report, not peer-reviewed. Sample of over 10,000 developers across 1,255 teams. <https://www.faros.ai/blog/ai-software-engineering>
- [11] S. E. Farrag. 2026. The Productivity-Reliability Paradox: Specification-Driven Governance for AI-Augmented Software Development. arXiv:2605.01160, preprint, not yet peer-reviewed. <https://arxiv.org/abs/2605.01160>
- [12] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler. 2021. The SPACE of Developer Productivity: There's More to it than You Think. *ACM Queue* 19, 1 (2021), 20–48. <https://doi.org/10.1145/3454122.3454124>
- [13] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin. 2025. The Current Challenges of Software Engineering in the Era of Large Language Models. *ACM Transactions on Software Engineering & Methodology* 34, 5 (2025), 1–30. <https://doi.org/10.1145/3712005>
- [14] S. G. Hart. 2006. NASA-Task Load Index (NASA-TLX); 20 years later. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, Vol. 50. 904–908.
- [15] S. G. Hart and L. E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Human Mental Workload*, P. A. Hancock and N. Meshkati (Eds.). North-Holland, 139–183.
- [16] Arshia Hemmat, Mohammadreza Sharbaf, Shekoufeh Kolahdouz-Rahimi, Kevin Lano, and Sobhan Y. Tehrani. 2025. Research directions for using LLM in software requirement engineering: a systematic review. *Frontiers in Computer Science* 7 (2025). <https://doi.org/10.3389/fcomp.2025.1519437>

- [17] ISO/IEC. 2023. *Information technology - Artificial intelligence - Management system*. International Standard ISO/IEC 42001:2023. International Organization for Standardization/International Electrotechnical Commission.
- [18] H. Kniberg and A. Ivarsson. 2012. *Scaling Agile @ Spotify with Tribes, Squads, Chapters & Guilds*. Whitepaper, industry-reported, not peer-reviewed. A description of Spotify's organisation at the time of writing, not a prescriptive method. <https://blog.crisp.se/2012/11/14/henrikkniberg/scaling-agile-at-spotify>
- [19] Leading Complexity. 2023. Summary of Henrik Kniberg's session, about his thoughts on the Spotify Model, in the Leading Complexity programme 2022. Crisp's Blog. Third-party published summary of Henrik Kniberg's session in the Leading Complexity programme 2022, not written by Kniberg himself. <https://blog.crisp.se/2023/02/06/leadingcomplexity/leading-complexity-series-summary-1-henrik-knibergs-thoughts-on-the-spotify-model>
- [20] J. Lee. 2020. Failed #SquadGoals. Practitioner account by a former Spotify employee, industry-reported, not peer-reviewed. <https://www.jeremiahlee.com/posts/failed-squad-goals/>
- [21] National Institute of Standards and Technology. 2023. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Technical Report NIST AI 100-1. National Institute of Standards and Technology, U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.100-1>
- [22] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590, preprint, not published in a peer-reviewed venue. <https://arxiv.org/abs/2302.06590>
- [23] J. Sauro and J. R. Lewis. 2016. *Quantifying the User Experience: Practical Statistics for User Research* (2nd ed.). Morgan Kaufmann.
- [24] S. Siddeeq, M. Abbasi, J. Rasku, Z. Zhang, F. Christophe, T. Mikkonen, and P. Abrahamsson. 2026. Epic-Organized vs. Requirement-Aligned Gherkin: An Empirical Evaluation of LLM-Based Acceptance Criteria Generation. Accepted at the International Conference on Software Engineering of Emerging Technologies (SEET 2026), Springer Nature. arXiv:2607.01980. <https://arxiv.org/abs/2607.01980>
- [25] M. Skelton and M. Pais. 2019. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press, Portland, OR.
- [26] Stack Overflow. 2025. 2025 Developer Survey: AI. Annual practitioner survey, industry-reported, not peer-reviewed. <https://survey.stackoverflow.co/2025/ai/>
- [27] Thoughtworks. 2025. Spec-driven development. Technology Radar, Vol. 33. <https://www.thoughtworks.com/en-us/radar/techniques/spec-driven-development>
- [28] Veracode. 2025. 2025 GenAI Code Security Report: Assessing the Security of Using LLMs for Coding. Industry report, not peer-reviewed. Assessment of over 100 large language models across Java, Python, C# and JavaScript. https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf
- [29] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, and Z. Zhuang. 2025. Empowering Agile-Based Generative Software Development through Human-AI Teamwork. *ACM Transactions on Software Engineering & Methodology* 34, 6 (2025), 1–46. <https://doi.org/10.1145/3702987>
- [30] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li. 2025. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. arXiv:2512.03262, preprint, not yet peer-reviewed. <https://arxiv.org/abs/2512.03262>